

Authority-Aligned Linearization for Rescue-Service Incident Response over Intermittent Edge Networks

Anonymous Author(s)

Anonymous Affiliation

Submitted to NCA 2026 (double-blind review)

Abstract—In rescue-service incident response, one principal holds operational command authority at every moment, yet responders must keep working through responder-side and backhaul partitions. Authority-bearing decisions must therefore stay attributable to that single commander even under intermittent connectivity. Standard distributed-systems tools do not fit this setting. Consensus moves the writer to whatever quorum is reachable, and conflict-free replicated data type (CRDT) merge is mergeable-by-default. Neither lets an operator designate the authoritative writer under partition. We present authority-aligned linearization (AAL), a consistency discipline. AAL places the incident-journal linearization point at the edge node that currently holds command authority, so authority-bearing writes serialize at that node. Non-authority data (reference state, field submissions, materialized views) stays locally available and is forwarded later, and command transfer uses fenced promotion rather than leader election. We specify and verify AAL using TLA+ and Hypothesis and implement it in a prototype built around responder outboxes, a single incident journal, and fenced promotion. We evaluate it under *tc-netem* emulated partitions against a Hanssen-style CRDT-LWW baseline and a Raft consensus baseline. The evaluation confirms the gap concretely: CRDT merge cannot encode authority for non-commutative decisions, and Raft moves the writer to the reachable quorum rather than the designated commander. Under AAL, partitions on either responder or backhaul links do not block command-edge progress. A fenced promotion’s protocol cost is single-digit milliseconds; by design, wall-clock time is dominated by the operator’s confirmation. The cost is no automatic failover.

Index Terms—authority-aligned linearization, network architectures and protocols, edge computing, intermittent connectivity, distributed systems, incident response, prototype evaluation

I. INTRODUCTION

Public-safety information systems must remain operational when commercial connectivity fails [1]–[3]. Rescue-service vehicles operate up to several hours away from regional dispatch centres. In that terrain, wireless connectivity alternates between full wide-area coverage, mesh-only operation, and total isolation [4]–[6]. Disasters routinely degrade or destroy commercial telecom infrastructure precisely when coordination demand is highest [7]. As documented in the 2018 Swedish forest-fire crisis [8], vehicles must continue acting on stale or partial information, and field submissions entered on tablets must survive hours without connectivity. In command-hierarchical environments the consequences of information loss are wrong assignments, missed hazards, and conflicting orders [9].

The operational constraints in this paper are grounded in public Swedish rescue-service guidance: Räddningstjänsten

Storgöteborg’s incident-planning and municipal rescue-plan guidance [10], [11]. That guidance treats incident-planning information as an ongoing, secrecy-restricted process in operational mobile IT support, not a static paper artifact. Independent academic studies of how Swedish rescue services use information systems confirm these end-user requirements and the gaps in information flow [12], [13]. From these public sources we derive three requirements and two deployment constraints. The requirements are R1: offline operation, R2: durable forward-only field submission, and R3: attributable command authority. The deployment constraints are avoiding cross-vehicle broadcast hazards and reusing existing rescue-service certificate and identity infrastructure.

The design question is: *how should authority-bearing data be synchronized across intermittently connected edge tiers, when the operational model designates a single principal as authoritative?* The answer is authority-aligned linearization (AAL), a consistency discipline. AAL places the incident-journal linearization point at the command edge, the vehicle edge node that currently holds operational command authority. We specify and verify AAL using TLA+ [14] and Hypothesis [15] and implement it in a prototype built around responder outboxes, a single incident journal, and fenced promotion.

We make three contributions:

- 1) (C1) a problem analysis of authority-bearing incident coordination under intermittent connectivity, showing why neither local-first, CRDT-based, nor consensus-based approaches directly satisfy rescue-service requirements for attributable command authority;
- 2) (C2) an authority-aligned synchronization model based on a single authoritative incident journal and fenced promotion, formally specified in TLA+ and mirrored by Hypothesis state-machine tests, verifying key authority invariants;
- 3) (C3) an executable prototype (Docker Compose, *tc-netem*) and partition-based evaluation against Hanssen-style CRDT-LWW and Raft baselines, demonstrating the feasibility and operational characteristics of AAL under intermittent connectivity.

II. AUTHORITY-BEARING INCIDENT COORDINATION UNDER INTERMITTENT CONNECTIVITY

Rescue-service incident response exposes a design tension between availability and authority. Responders must continue operating under intermittent connectivity, yet authority-bearing

incident decisions must remain attributable to a single operational commander.

The target setting is municipal rescue-service incident response, where a regional core supports dispatch and reference data, vehicle edge nodes operate near the incident site, and field tablets attach to local vehicles acting as leaf clients. Connectivity is intermittent, vehicles may move between wide-area connectivity, mesh-only contact, and total isolation [4], [5] (Figure 1). One vehicle edge node holds the command role for the incident; we refer to it as the *command edge*. The role can move to another vehicle, but only through an explicit human decision, and the protocol must respect that operational authority.

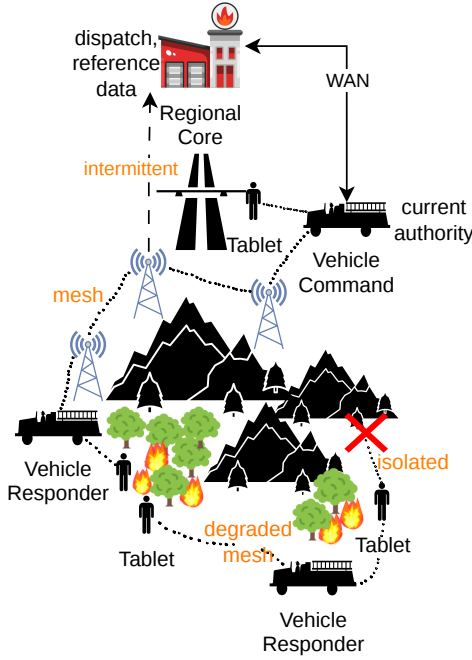


Fig. 1. Operational setting. A regional core supports dispatch and reference data; vehicle edge nodes run near the incident site; tablets attach to a local vehicle. One vehicle holds command authority at any instant. Vehicles simultaneously occupy different connectivity regimes: wide-area backhaul, intermittent backhaul with mesh transit, or total isolation.

A. Operational Requirements

Public rescue-service guidance and studies of Swedish rescue-service information systems motivate the requirements used here [10]–[13]. R1 and R2 follow from mobile-IT and incident-reporting findings that responders need locally available information and durable later documentation [12], [13]. R3 follows from decision-provenance requirements and rescue-service command guidance [10], [11], [16].

The externally motivated operational requirements are:

- **R1 (Offline operation)** Each tier must keep working from previously synchronized data without continuous connectivity.
- **R2 (Durable forward-only submission)** Field submissions entered on tablets must survive vehicle restart and prolonged partition.

- **R3 (Attributable command authority)** Resource assignments, status transitions, and hazard annotations must be attributable to one operational command authority at any instant.

These requirements sit at different layers, by design. R1 and R2 are availability and durability properties that AAL satisfies with weaker local and eventual contracts, outside the authority boundary; R3 is a single-authority property satisfied inside it. The contribution is the M1 state-class partition that lets the three be met by different contracts, rather than forcing one contract on all incident data.

R3 is not a claim that crisis governance is simple. Incident-command studies and emergency-management reviews describe hierarchies embedded in multi-actor response networks, where authority may be shared, delegated, contested, or legally bounded [17]–[19]. Yet emergency-response decision provenance still requires a defined subset of command decisions to remain attributable to the authority that made them [16]; R3 abstracts that accountability property, authority-bearing writes have one current command authority, while other coordination need not be single-leader. For AAL, this yields conservative transfer semantics, a new command edge is accepted only after operator-initiated promotion, not inferred from reachability.

B. Positioning Relative to Consensus, CRDTs, Local-first, and Emergency Networking

AAL recombines known mechanisms—single-leader replication, transactional outbox [20], idempotency keys, and manual fencing—under a domain rule: the command edge is the incident-journal linearization point. The incident-state entity owns its decision history; external actors submit forward-only events rather than mutate it directly [21]. The contribution is the M1 state-class partition and grounding the boundary in external command authority; the mechanisms themselves are not claimed as novel.

Consensus and replicated state machines. Consensus, shared-log, and chain-replication systems provide replicated agreement and totally ordered logs under their stated failure models [22]–[25]. These systems supply stronger ordering than R3 requires, at a cost the workload does not justify: rescue-service traffic is dominated by reference-data lookups and append-only reporting, for which quorum agreement and leader election are over-engineered. Election-based liveness selects a writer by reachability, whereas R3 fixes the writer by operational command; § IV-E1 measures this mismatch on a three-voter Raft baseline.

CRDTs and strong eventual consistency. CRDT and strong-eventual-consistency work provides convergence for mergeable replicated state [26]–[28]. It serves R1/R2 well but cannot satisfy R3 [29]. Authority-bearing incident decisions such as resource assignments, status transitions and hazard annotations are non-commutative: there is no application-defensible merge for two responders setting the same status to conflicting values. R3 therefore requires attribution and a single authoritative order, not automatic conflict rejection. AAL’s measured path preserves both conflicting writes in an attributed total order;

semantic guards are a disclosed extension (§ V-B). Any deterministic merge rule implicitly invents the authority R3 requires be explicit.

Offline-first and local-first systems. Offline-first and local-first systems prioritize local availability and continued operation under poor connectivity [20], [30]. They satisfy R1 and R2 and are the right model for reference data, field submissions, and materialized views. But their design goal is replica *autonomy*—every node is equally entitled to write and reconcile—which is precisely the property the command hierarchy rejects for authority-bearing decisions (R3).

Why not a centralized backend? A regional backend cannot hold the linearization point when it is partitioned from the incident, which is the common case: vehicles operate hours from dispatch (§ I), and no server can serialize writes it cannot reach. Authority-bearing command decisions must therefore serialize at the incident during total backhaul isolation. A backend-authoritative offline-first design has two bad options. It can block command decisions while isolated, violating R1 for the command stream. Or it can let field nodes write provisionally and reconcile later, reintroducing the multi-writer merge problem R3 forbids. AAL places the linearization point at the command edge precisely so the authoritative order survives backhaul partition; the regional core still owns reference data, which tolerates the weaker R1 and R2 contracts.

Emergency and edge networking. Closest in setting is emergency and edge networking. CoNICE [31] reaches consensus among infrastructure-less peers using One-Third-Rule agreement, layered over epidemic replication, causality metadata, and decision invalidation for long-term fragmentation. That is substantial machinery when the operational problem already designates a commander: rescue-service incident command has a designated commander at every moment, and the network is not permitted to vote on incident decisions, so CoNICE’s no-privileged-peer assumption is exactly the one R3 removes. Public-safety networking and edge/fog research place resilient transport and storage near responders [5], [32], [33], addressing R1 transport but not which actor may issue an authority-bearing decision.

Across these families, each helps responders keep working or order writes, but none decides *which actor* may issue an authority-bearing decision during a partition. Primary/backup systems already use manual failover and fencing tokens [34]; the distinction here is using operational command authority to set the linearization boundary while assigning weaker contracts to non-authority incident data.

C. Assumptions and Design Choices

These choices are responses to the requirements, not the requirements themselves. R3 does not require a single-writer database for all incident state; it requires authority-bearing decisions to remain attributable to the current command authority. AAL conservatively places the incident-journal linearization boundary at the command edge, while reference data, field submissions, and materialized views keep weaker local or eventual contracts.

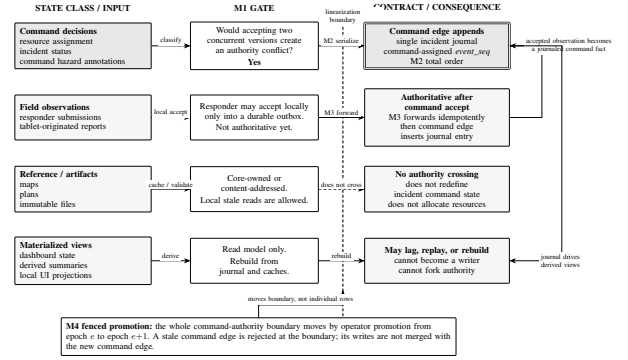


Fig. 2. AAL linearization-boundary/state-class partition. M1 places the single-writer authority boundary at the command edge: command decisions are serialized into the incident journal by M2 with command-assigned *event_seq*. Outside the boundary, reference/artifact data, responder outboxes, and materialized views use local/eventual contracts for R1/R2; M3 forwards durable, idempotent field observations across the boundary, becoming authoritative only after command acceptance. M4 moves the boundary by fenced operator promotion, so the boundary is set by operational command rather than Raft-style reachability or CRDT-style merge.

The deployment constraints identified in the introduction are separate. The mesh should act as routed transit rather than a stretched user network, and the system must reuse the rescue-service identity infrastructure. Their architectural consequences—edge cache and local services, tablet/edge outbox and retry, a single incident-journal writer, mesh-as-transit routing, and mTLS/CA identity integration—are developed in § III.

III. AUTHORITY-ALIGNED ARCHITECTURE AND PROTOCOL

The requirements place conflicting demands on incident data. R1 and R2 require local work and durable submission during disconnection; R3 requires one authoritative order for command decisions. AAL handles that split with four mechanisms; M1 partitions state, M2 orders command writes, M3 carries responder submissions, and M4 moves command authority without forking the journal.

A. State Classes and Consistency Contracts (M1)

M1 sets the consistency boundary for R1, R2, and R3. Non-authority classes use local/eventual contracts for R1 and R2; authority-bearing decisions use the single-writer contract R3 demands. An operation crosses the linearization boundary only when accepting concurrent versions would create an authority conflict, like resource assignments, incident status transitions, and command hazard annotations. These are accepted only by the command edge and inserted into the incident journal with a command-assigned *event_seq*.

This is the paper’s RedBlue-style partition [35]; the “red” set is the authority-bearing command decision stream, not the whole incident database. Figure 2 is the visual definition of M1, the authority boundary follows command authority, not reachability or merge.

Outside the boundary, edges still work while links are intermittent: reference data are core-owned and cached, so

TABLE I

ONLY AUTHORITY-BEARING JOURNAL ENTRIES USE THE SINGLE-WRITER CONTRACT; OTHER INCIDENT STATE USES LOCAL OR EVENTUAL SYNCHRONIZATION CONTRACTS.

State	Authority/source	Synchronization contract
Reference data	regional core	One-way core-to-edge replication; eventual convergence.
Immutable artifacts	publisher / core	Manifest, package, or hash validation where implemented.
Field observations	tablet / responder	Locally durable outbox; at-least-once forwarding; idempotent accept.
Command decisions	command edge	Single-writer rule applies only here; total order by <i>event_seq</i> .
Materialized state	journal	Derived and rebuildable; not authoritative.
Audit events	edge / core	Eventual forwarding; validation incomplete.

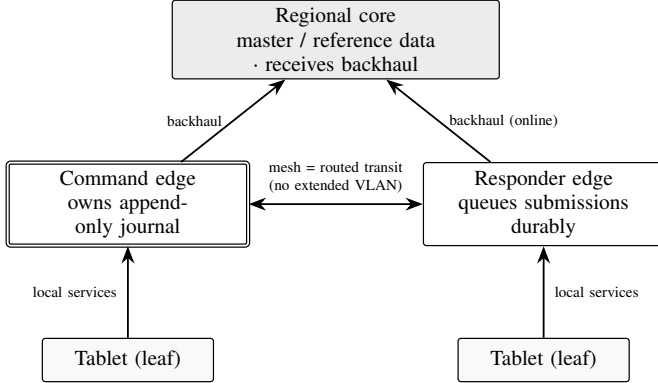


Fig. 3. Tiered edge topology. The regional core holds master/reference data and receives backhaul; vehicle edges run local services, with the command edge owning the append-only incident journal and responder edges queuing submissions; tablets are leaf clients. The mesh between vehicles is routed transit, not a consistency mechanism, and user VLANs are not extended across vehicles.

partitions permit stale local reads, not local redefinition, and immutable artifacts, audit records, and command-to-core backhaul use validation or eventual forwarding rather than command-edge linearization. Moving command authority requires the fenced-promotion rule of § III-E so a stale command edge cannot keep appending; Table I summarizes these contracts.

B. Tiered Edge Topology

Vehicle edges provide local services during disconnection; tablets are leaf clients outside command sequencing (Figure 3).

After M1 fixes which operations need command ordering, the protocol handles three cases: command writes crossing the boundary, responder submissions reaching it, and command transfer moving it.

The main identifiers are *incident_id*, client-supplied *client_event_id*, command-assigned *event_seq*, and, for promotion, a command epoch (a numbered tenure; higher means newer authority).

C. Incident Journal Sequencing (M2)

At that boundary, M2 lets command keep writing during responder outages without weakening R3. The command edge

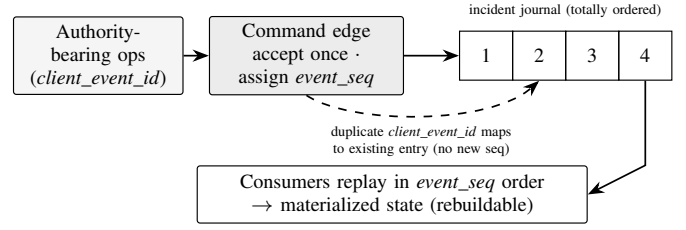


Fig. 4. M2 incident-journal sequencing. The command edge accepts each *client_event_id* once and assigns *event_seq* atomically with journal insertion, producing a totally ordered log; a duplicate *client_event_id* folds onto its existing entry rather than taking a new sequence number. Consumers replay in *event_seq* order, and materialized state is rebuildable from the journal.

linearizes authority-bearing operations into a totally ordered journal (Figure 4).

D. Durable Outbox and Idempotent Submission (M3)

For R1 and R2, M3 gives responders a safe local path before command is reachable: non-authority data are locally durable, idempotent, and eventually forwarded to the incident journal. Figure 5 shows the steady-state field-submission path. The boundary is between local durability and authoritative command commit. A responder may acknowledge local acceptance after the outbox insert and mark an item forwarded only after incident-journal acceptance or duplicate recognition.

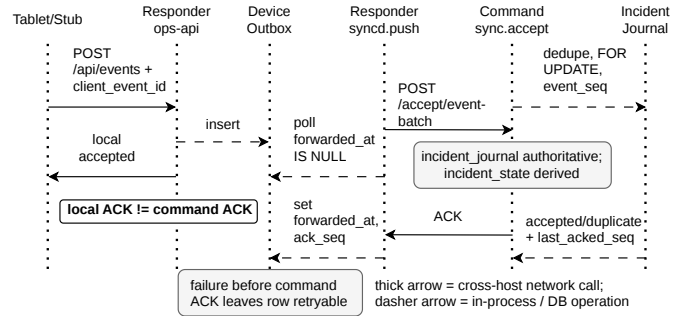


Fig. 5. Responder outbox protocol: the steady-state field-submission path from tablet to incident journal. The journal assigns the authoritative sequence number, and duplicate delivery is handled by *client_event_id*.

The M3 contract has three parts:

- 1) Each submission has a globally unique idempotency key, *client_event_id*, scoped by the incident and originating client identity.
- 2) A responder accepts a submission only after the event is stored in its local outbox.
- 3) A responder may mark an outbox item forwarded only after the command edge acknowledges durable incident-journal acceptance or duplicate recognition.

Outbox delivery is at least once and is deduplicated at the command edge; there is no claim of causal ordering across independent responder outboxes before command assignment.

E. Fenced Promotion (M4)

When command authority moves, M4 keeps R3 intact. Authority transfers without forking the journal. The design does not provide autonomous failover. An operator confirms fenced promotion, and fencing prevents stale command authority from continuing.

Figure 6 summarizes the intended rule. Each command tenure has a monotonically increasing epoch or equivalent term; promotion emits an explicit record; stale command traffic must be rejected once a newer epoch is known. This strict rule sacrifices availability when the previous command cannot be fenced or its status cannot be established. In the model this rule preserves *SingleAuthority* and *NoForkedJournal*; in the prototype, rejecting a stale epoch (median 2.5 ms) is measured on the service path and is cheaper than a commit. § IV-D reports the strict-versus-weak comparison.

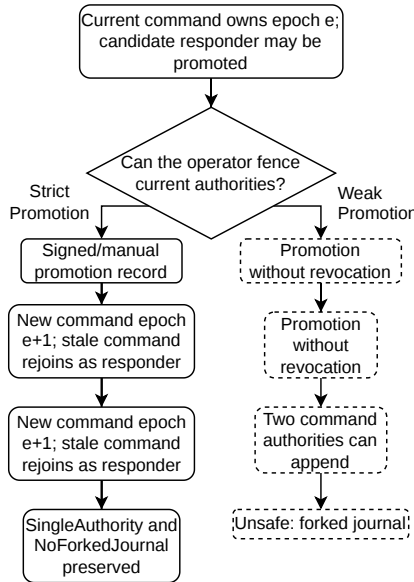


Fig. 6. Fenced promotion preserves the single-writer invariant. Left (single-border): with fencing, the stale command is forced to rejoin as a responder. Right (double-border): without fencing, two command authorities can append concurrently, producing a forked journal. The unsafe branch appears as a 3-state TLA+ trace for split authority and a 5-state trace for a forked journal; durable epoch storage and stale-epoch rejection are implemented in the prototype.

F. Partition Behavior and Promotion Blocking

During a responder-command partition, the responder satisfies R1 locally with cached data and outbox storage, but submissions are not authoritative until the command edge accepts them and assigns *event_seq*. During a command-core partition, R1 holds at command: incident work proceeds and the core catches up later from the journal. If command moves to another vehicle, R3 rules out automatic election: an operator must promote the new command edge and prevent stale command authority from continuing to write, otherwise promotion is blocked.

IV. PROTOTYPE AND EVALUATION

A. Prototype and Emulation Setup

The artifact repository contains a prototype service path for the core and edge tiers, plus deployment scaffolding for the target vehicle environment. Python services target Python 3.12 and use FastAPI and *asyncpg*. Docker Compose files instantiate the emulated core, command edge, and responder edges used below; a separate *baseline/crdt* stack provides the CRDT-LWW comparison point.

Table II records the prototype evidence boundary for each protocol concept from § III. We use three evidence levels throughout: *measured* (exercised on the prototype service path), *model-level* (established by the TLA+/Hypothesis checks but not the service path), and *deployment-only* (target-deployment design, not exercised here).

TABLE II
PROTOTYPE EVIDENCE BOUNDARY FOR EACH PROTOCOL CONCEPT.

Concept	Status
<i>M1–M2: Linearization boundary and incident-journal sequencing</i>	
Incident journal and <i>event_seq</i>	Measured on command-edge <i>syncd</i> accept.
Command-edge accept during responder partition	Measured on direct command-edge <i>syncd</i> accept during responder isolation; command <i>ops-api</i> not exercised.
<i>M3: Durable outbox and idempotency</i>	
Responder durable outbox	Measured for responder forwarding.
<i>client_event_id</i> idempotency	Measured by duplicate replay (§ IV-C).
<i>M4: Fenced promotion</i>	
Promotion and fencing	Service-level command epochs and stale rejection; strict promotion in TLA+; two-edge handoff in Hypothesis; stale rejection measured in § IV-D.
<i>Supporting infrastructure (not authority-aligned)</i>	
Core bootstrap and backhaul	Synthetic bootstrap/backfill; evaluated in Docker.
Vehicle edge roles	Command and responder edges in Compose.
Incident-journal forwarding to core	Measured for WAN recovery/backfill.
Tablet client	Python stub in evaluation; Android scaffold only.
Multi-condition netem evaluation	Measured under two link profiles in § IV-C.
Transit and security controls	Deployment design only; physical mesh and security overhead are out of scope.

Security controls (VPN overlay, mTLS termination, certificate-derived identity) are deployment design, not measured here; the emulation uses plain HTTP on a Docker bridge with synthetic device and user identifiers. We evaluate the prototype on a single-host emulation, organized by the four mechanisms of § III (M1–M2, M3, and M4). Each subsection names the requirement it serves, the claim it tests, and the evidence level—*measured*, *model-level*, or *deployment-only* (§ IV). Three further experiments delimit what AAL does not promise rather than evaluate a mechanism: a Raft consensus baseline (§ IV-E1), a CRDT-LWW semantic falsification (§ IV-E2), and command-to-core WAN backhaul recovery (§ IV-E3).

Measurements come from a Docker Compose emulation with one regional core, one command edge, and three responder edges. The propagation, recovery, and throughput cells use a single responder path, so reported latencies are per-path, not

TABLE III

SERVICE-PATH MEASUREMENTS FOR RESPONDER PROPAGATION, WAN RECOVERY, COMMAND SERIALIZATION, AND RESPONDER ISOLATION.

Path / cell	Result	Interpretation
<i>Responder-to-command propagation</i>		
queue depth = 1 ($n = 30$)	median 710.1 ms; p95 832.1 ms	outbox-to-command path journal
queue depth = 10 ($n = 30$)	median 639.4 ms; p95 719.5 ms	outbox-to-command path journal
queue depth = 100 ($n = 30$)	median 931.5 ms; p95 1045.5 ms	outbox-to-command path journal
<i>Command-to-core WAN recovery</i>		
partition = 1 s ($n = 30$)	median 0.75 s; p95 0.92 s	support-path backhaul, not command authority
partition = 10 s ($n = 30$)	median 0.91 s; p95 1.04 s	support-path backhaul, not command authority
partition = 60 s ($n = 30$)	median 0.23 s; p95 1.05 s	support-path backhaul, not command authority
<i>Command journal serialization</i>		
direct /accept ($n = 29$)	median 356.3 events/s; p95 491.7	linearization point: command accept + journal write only
end-to-end via outbox ($n = 29$)	median 159.4 events/s; p95 163.9	full responder outbox → push-batch → command path
<i>Command commit during responder isolation</i>		
direct command accept path ($n = 20$)	20/20 commits; seq. 1-20; 0 duplicate seqs.	direct /accept/event-batch path; median/p95 latency 5.76/7.32 ms

aggregate. Each cell repeats $n = 30$ unless noted, with median and p95 reported. Partitions cut only the targeted path with *tc*, so each cell isolates one effect. Impairment uses two *tc-netem* profiles informed by published mesh measurements [4], [32]: stressed (80 ms $\pm$ 30 ms delay, 2 % loss) and degraded (200 ms $\pm$ 80 ms delay, 8 % loss). Table III collects the primary service-path numbers.

B. M1–M2: linearization boundary and journal sequencing

M1–M2 serve R3, and R1 for command-side writes. Claim: authority-bearing writes commit independently of responder reachability and receive consecutive, non-duplicate *event_seq* values. Evidence: measured.

With one responder isolated for ten seconds, direct single-event POSTs to */accept/event-batch* all commit, assigning consecutive non-duplicate *event_seq* values at single-digit millisecond latency (Table III). The endpoint requires a current *X-Command-Epoch*; stale or absent epochs are rejected before journal work. Responder reachability is therefore not in the commit path.

Saturation throughput at the linearization point (Table III) reflects a single-writer serializer. Even a generous 10 authority-bearing events per minute is 0.17 events/s, so the measured 356.3 events/s exceeds plausible incident demand by three orders of magnitude.

C. M3: durable outbox and idempotent submission

M3 (the durable responder outbox with *client_event_id* idempotency) serves R1 and R2. Claim: responder submissions

TABLE IV

RESPONDER PROPAGATION UNDER *tc-netem* IMPAIRMENT, 20 MEASUREMENTS PER CELL; ALL CELLS COMPLETE WITH NO DROPS.

Profile	Queue depth	Median	p95
Stressed	1	1215 ms	1494 ms
Stressed	10	2860 ms	3355 ms
Degraded	1	3647 ms	5329 ms
Degraded	10	8104 ms	10895 ms

reach the journal exactly once under retry, process restart, and impairment typical of a mesh network. Evidence: measured.

Median propagation stays sub-second across queue depths 1–100; the p95 at depth 100 marginally exceeds 1 s (1045.5 ms, Table III).

Replaying 50 unique *client_event_id* values five times each leaves exactly 50 journal rows across $n = 30$ runs: forwarding is at-least-once and deduplicated at the journal, so submission is effectively exactly-once at the linearization point.

Across process restart, 50 outbox submissions survive a responder-Postgres power cycle before forwarding, then forward with no duplicates and consecutive *event_seq*, reaching journal consistency about 2.6 s after recovery. This is the runtime witness for R2.

Under both *tc-netem* profiles, all 80 measurements (20 per cell at queue depths 1 and 10) complete with no drops; latency degrades gracefully, with the heaviest tail at degraded qd-10 (median 8.1 s, p95 10.9 s, Table IV).

D. M4: fenced promotion

M4 (fenced promotion with stale-epoch rejection) serves R3 under command transfer, exercising the fenced-promotion mechanism (C2). Claim: a fenced promotion preserves single authority across an epoch change and rejects stale-epoch traffic before any incident-journal work. Evidence: stale-epoch rejection is measured on the service path; multi-edge *SingleAuthority* and *SingleCommand* are model-level because the service harness is single-edge and lacks core-to-edge journal bootstrap.

Protocol cost is single-digit milliseconds across all three phases of a transition (Figure 7), measured over $n = 300$ events (100 per phase). Commit phases A and C have medians 6.3 ms (p95 14.0 ms) and 5.7 ms (p95 10.7 ms); the phase B stale-rejection path is 2.5 $\times$ cheaper (median 2.5 ms, p95 5.6 ms) by construction, since the fence rejects stale traffic before the journal write, so the saving is a skipped write. Operator confirmation sits in the critical path by design and dominates wall-clock; the protocol contribution is orders of magnitude smaller, an operator cost rather than a network cost.

In five live service-path runs, phase A commits 20/20 events at epoch 1, phase B returns 20/20 HTTP 409 rejections for the stale epoch and adds zero incident-journal rows, and phase C commits 20/20 events at epoch 2. The incident journal holds 200 rows total, split 100/100 between epochs, with no *event_seq* value appearing under both.

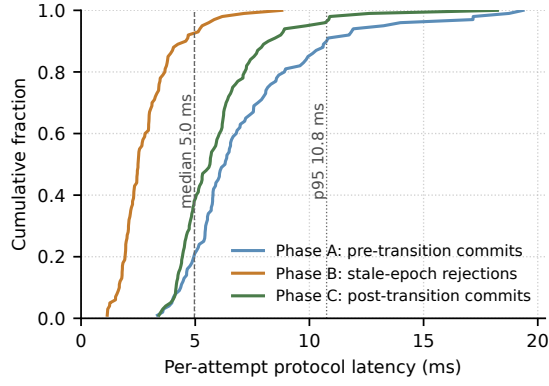


Fig. 7. Fenced-promotion protocol cost by phase. Phase A is pre-transition commits at epoch e , phase B is stale-epoch rejections at the boundary, phase C is post-transition commits at epoch $e + 1$. $n = 300$ events (100 per phase). Vertical markers are pooled across phases: median 5.0 ms, p95 10.8 ms. The cost shown is the protocol path only; the operator-confirm step is excluded by design.

TABLE V

FALSIFICATION-COST ASYMMETRY. THE STRICT VARIANT VERIFIES SIX SAFETY INVARIANTS OVER TENS OF MILLIONS OF DISTINCT STATES; EACH WEAK VARIANT PRODUCES A COUNTEREXAMPLE IN HUNDREDS TO THOUSANDS OF DISTINCT STATES UNDER A SECOND OF WALL-CLOCK.

Variant	Outcome	States	Time
strict	all six invariants pass	38.56 M	13.0 min
weak	<i>SingleAuthority</i> violated	185	0.85 s
weak	<i>EpochAuthorityCoupling</i> violated	144	0.77 s
weak	<i>NoForkedJournal</i> violated	1.0k	0.83 s

Table V contrasts the cost of verifying the strict variant with the cost of falsifying the weak ones. The strict configuration (3 vehicles, journal bound 7, 7 client-event IDs, 3 fenced promotions, defined as state-space caps in the TLA+ comments) verifies six safety invariants over 38.56 M distinct states in 13 minutes; each weak variant instead produces a counterexample of trace length 3–5 in under a second. Python Hypothesis tests mirror the strict/weak model over 2000 examples of length 50 and add a two-command-edge handoff model over 1000 examples of length 40, checking forwarded-prefix bootstrap, stale-authority rejection before journal mutation, and suffix extension on the promoted edge. No strict violation appears, and weak variants have on-demand counterexamples.

E. Scope-delimiting paths

1) *Raft leadership follows the reachable quorum*: This tests the R3-relevant mismatch between consensus leadership and operational command authority. The baseline is the three-voter *hashicorp/raft* journal-commit cluster in *baseline-raft*, whose in-memory FSM assigns *event_seq* values with *client_event_id* deduplication. For each behavioral run, the harness records the current leader L_0 as the node holding write authority and hard-cuts it from the other two voters; the reachable majority then elects L_1 and accepts writes there (Table VI), with election timing randomized across unseeded runs. This is

correct Raft behavior, authority moves to a node that can form a quorum [23], and it is exactly the mechanism mismatch for R3: Raft has no input by which an operator can pin the writer to an externally designated command edge.

2) *CRDT semantics fail for authority-bearing data*: This tests C1: mergeable-by-default replication cannot encode authority for non-commutative incident decisions. It is a semantic falsification, not a speed comparison.

The CRDT-LWW baseline is four FastAPI replicas using Hanssen’s operation-based Last-Writer-Wins Element Set [27]. It preserves local acceptance but loses authority-bearing conflicts: concurrent status updates converge to the latest timestamp, and a later update resurrects a deleted element. LWW discards the losing status from materialized state, whereas AAL preserves both writes in an attributed total order. Ordering and attribution are R3’s service-path scope; semantic rejection is an additional guard disclosed in § V-B.

The baseline is kept deliberately lightweight, in-memory, with no background anti-entropy, converging only through harness-triggered *full_sync()* after traffic stops. This is because the test does not require storage parity; it requires only that the merge rule be exercised on concurrent authority-bearing writes. The asymmetry favors CRDT where it already performs better, which is the correct bias for a falsification: if mergeable-by-default semantics cannot encode authority even under favorable conditions, the semantic argument holds regardless of infrastructure differences. Table VII keeps measured AAL behavior, model-level AAL safety, and measured CRDT-LWW semantics separate.

3) *Backhaul recovers within about a second of WAN heal*: This measures command-to-core forwarding after a WAN heal, with a narrowly targeted *tc* cut of the command-to-core path only, so the measurement isolates backhaul flush time (Table III). Recovery is bounded by the forward poll loop and backlog, not outage length: the forwarder polls every 1.0 s while forwarding and every 2.0 s after empty cycles, so non-monotonic medians across 1, 10, and 60 s outages are expected poll-phase effects.

Together, these results place the authority boundary where § III does; § V-B details the resulting limitations and non-claims.

V. SCOPE AND LIMITATIONS

A. Threat Model and Security Scope

The target deployment assumes an organizational CA or identity provider with enrolled devices and users; the backend trusts certificate-derived identity only when the reverse proxy and service boundary are correctly configured. It handles replayed field submissions through *client_event_id* idempotency and limits tablet reachability through local edge APIs and network segmentation.

Several threats remain outside the prototype evaluation. A stolen or compromised tablet can be revoked only when nodes regain contact with the authority infrastructure or receive an updated trust bundle, so instantaneous revocation during total disconnection is not provided. A stolen vehicle node is

TABLE VI

RAFT CONSENSUS BASELINE COMPARISON. THE PARTITION RESULT IS THE R3-RELEVANT FINDING; AUTHORITY FOLLOWS THE REACHABLE QUORUM RATHER THAN AN OPERATOR-DESIGNATED COMMAND EDGE.

Scenario	Measured result	Interpretation
Event sequencing sanity	leader batch accepted 1 event, deduplicated 1, last seq. 1; journal count 1	Confirms the Raft FSM assigns <i>event_seq</i> and enforces <i>client_event_id</i> idempotency.
Leader-minority partition	12/12 runs elected $L_1 \neq L_0$; 12/12 committed 5 writes at L_1 ; new-leader median 2.23 s, p95 3.02 s	L_0 writes while isolated: stall then apply timeout 12/12; authority follows the reachable quorum, not an external command designation.

TABLE VII

SEMANTIC COMPARISON WITH A HANSSSEN-STYLE OPERATION-BASED CRDT-LWW BASELINE, SEPARATING MEASURED SERVICE-PATH RESULTS FROM MODEL-LEVEL AAL GUARDS.

Scenario	AAL result	CRDT-LWW result	Interpretation
Field-edit propagation	AAL bridge median 639.4 ms at qd=10; 931.5 ms at qd=100	CRDT qd=10 stressed/degraded 814.1 ms/2.54 s; qd=100 degraded 3.97 s	Both preserve durable propagation; CRDT anti-entropy is not an authority boundary.
Concurrent status edit	Current AAL service path sequences unique events; semantic rejection is model-level/not implemented in the measured prototype	30/30 LWW runs converge; winner counts contained: 30; median 12.8 ms	LWW accepts both writes and materializes only the latest value; losing op is only recoverable from the op log.
Delete/update race	Current AAL service path has no delete/update semantic guard; limitation disclosed	30/30 runs resurrect the element; median convergence 12.7 ms	Matches Hanssen’s stated LWW delete-semantics caveat.
No-partition throughput	AAL command-journal commit rate median 356.3 events/s (single writer)	CRDT aggregate local accept rate across three writers median 349.2 events/s	Different units; reports write-availability trade-off, not direct speedup.
Post-quiescence convergence (60 s outage, $n = 10$)	AAL command-to-core support-path recovery median 227.0 ms after a real tc WAN cut ($n = 30$)	CRDT all-replica convergence median 45.4 ms after anti-entropy resumes (no link impairment during the wait)	Comparable in outage duration, not transport mechanism; AAL keeps command-local ordering, CRDT converges once anti-entropy is exchanged.

more serious because it may hold cached data, and if it is the command edge it also holds operational authority. The architecture relies on device encryption, operational fencing, and audit rather than Byzantine tolerance: because authority is explicit, a captured or buggy command edge can poison the incident journal while it holds authority, a central trade-off of making command authority explicit.

The design also does not address a compromised organizational CA, Byzantine command behavior, physical tamper resistance beyond deployment controls, or audit-log integrity against a fully compromised node. These are deployment and hardening questions, not properties established by the emulation.

B. Evaluation Boundaries and Non-claims

The evidence for each invariant matches the appropriate verification method. *SingleAuthority* and *SingleCommand* express properties over all possible interleavings of concurrent authority claims—properties that model checking establishes exhaustively over 38.56 M distinct states but that a service-path test can only sample. The prototype’s service path is single-edge because core-to-edge journal bootstrap is not implemented, so multi-edge single authority is verified at the model level only; the two-edge Hypothesis handoff model covers prefix bootstrap and stale-authority rejection, and semantic rejection remains outside the prototype. *EpochAuthorityCoupling*, *NoForkedJournal*, *DurabilityAcrossPromotion*, and *LocalIdempotency* have both model verification and live service-path witnesses.

The prototype is scoped to measure protocol behavior under controlled partition conditions. The emulation isolates the authority, outbox, and fencing mechanisms from deployment variables—physical mesh transport, security overlay cost, and Android client integration—that do not bear on the authority model’s correctness. Accordingly, it does not claim physical Rajant mesh behavior, measured WireGuard or mTLS overhead, or end-to-end Android validation, and the workload is single-incident and limited-scale. The authority model’s safety properties are established by the TLA+ verification; the prototype demonstrates that the protocol can be implemented and operated under partition, and the Hypothesis state-machine tests check that the implementation’s behavior agrees with the model on the sampled traces.

VI. CONCLUSION

Authority-aligned linearization rests on an operational fact that consensus and CRDT designs do not directly address: at every moment, one principal is designated as authoritative. The incident-journal linearization point is the command edge, the vehicle edge node that currently holds that authority. Reference data, field submissions, materialized state, and audit forwarding sit outside that boundary on weaker contracts. Command transfer uses fenced promotion rather than leader election. Conventional offline-first synchronization does not provide this. A backend-authoritative design cannot hold the command-decision order under backhaul partition, the regime in which rescue-service incidents operate.

The TLA+ model establishes the safety boundary, while the prototype demonstrates its operational realization. Under

strict fencing, the model verifies single authority and no forked journal. The weak variant produces the expected split-brain witness, and the prototype shows command-edge writes proceed independently of responder reachability. The CRDT-LWW comparison shows the trade-off: CRDT replicas accept local writes at multiple nodes and converge quickly after anti-entropy exchange. AAL, by contrast, preserves one incident journal and treats semantic conflict rejection as a model-level boundary property. The cost is no automatic failover and lower write availability for authority-bearing operations.

Future work is to deploy the prototype on a vehicle-mounted mesh, exercise fenced promotion across multiple command edges, implement service-path semantic rejection for authoritative conflicts, and add crash-boundary and Android end-to-end tests. The harder comparison is empirical, not architectural: measure the design against CoNICE-class consensus and CRDT-based emergency systems on workloads where command authority is contested rather than designated.

REFERENCES

- [1] M. Wiśniewski, K. Szwarc, and W. Skomra, "Continuity of essential services as an emerging challenge for societal resilience," *IEEE Access*, vol. 11, pp. 44 614–44 635, 2023.
- [2] OECD, "Digital security and resilience in critical infrastructure and essential services," OECD Digital Economy Papers, Paris, 2019.
- [3] B. Karaman, I. Bastürk, S. Taskin, E. Zeydan, F. Kara, E. A. Beyazıt, M. Camelo, E. Björnson, and H. Yanikomeroglu, "Solutions for sustainable and resilient communication infrastructure in disaster relief and management scenarios," *IEEE Communications Surveys & Tutorials*, vol. 28, pp. 716–760, 2026.
- [4] E. G. Zimbelman, A. T. Hudak *et al.*, "Lost in the woods: Forest vegetation, and not topography, most affects the connectivity of mesh radio networks for public safety," *PLOS ONE*, 2022.
- [5] Q. Wang, S. Wang *et al.*, "An overview of emergency communication networks," *Remote Sensing*, 2023.
- [6] N. Andreaassen, O. J. Borch, E. Ikonen *et al.*, "Information sharing and emergency response coordination," *Safety Science*, 2020.
- [7] A. Marshall, C.-A. Wilson, and A. P. Dale, "Telecommunications and natural disasters in rural Australia: The role of digital capability in building disaster resilience," *Journal of Rural Studies*, vol. 100, p. 102996, 2023.
- [8] M. Murphy, "Digital transformation for crisis volunteerism: A study in the aftermath of the swedish forest fires crisis in 2018," Licentiate thesis, Linköping University, 2021. [Online]. Available: <https://www.diva-portal.org/smash/get/diva2:1570107/FULLTEXT01.pdf>
- [9] D. K. Allen, S. Karanasios, and A. Norman, "Information sharing and interoperability: The case of major incident management," *European Journal of Information Systems*, vol. 23, no. 4, pp. 418–432, 2014.
- [10] Räddningstjänsten Storgöteborg, "Råd och anvisning 201: Insatsplanering vid farlig verksamhet," Operational guidance document, 2023, revised 2023-10-25; valid from 2018-12-01. [Online]. Available: <https://www.rsgbg.se/globalassets/dokument/rad-och-anvisningar/rad-och-anvisning-201---insatsplanering-vid-farlig-verksamhet.pdf>
- [11] —, "Kommunal Plan för räddningsinsatser – Del I: Allmän del," Municipal rescue-services plan, 2025, dated 2019-01-02; revised 2025-06-09. [Online]. Available: <https://www.rsgbg.se/globalassets/dokument/foretag-och-organisationer/seveso/rsg-kommunal-plan-for-raddningsinsats-del-i---allman-del.pdf>
- [12] S. Pilemalm, D. Andersson, and K. Yousefi Mojir, "Enabling organizational learning from rescue operations: The Swedish rescue services incident reporting system," *International Journal of Emergency Services*, vol. 3, no. 2, pp. 101–117, 2014.
- [13] A. Westman *et al.*, "Mobilisation of emergency services for chemical incidents in Sweden – a multi-agency focus group study," *Scandinavian Journal of Trauma, Resuscitation and Emergency Medicine*, vol. 29, p. 99, 2021.
- [14] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.
- [15] D. R. MacIver, Z. Hatfield-Dodds, and many other contributors, "Hypothesis: A new approach to property-based testing," *Journal of Open Source Software*, vol. 4, no. 43, p. 1891, 2019.
- [16] I. Naja, L. Moreau, and A. Rogers, "Provenance of decisions in emergency response environments," in *Provenance and Annotation of Data and Processes*. Springer, 2010, pp. 221–230.
- [17] D. P. Moynihan, "Combining structural forms in the search for policy tools: Incident command systems in U.S. crisis management," *Governance*, vol. 21, no. 2, pp. 205–229, 2008.
- [18] I. Wennman, C. Jacobson, E. Carlström, A. Hyltander, and A. Khorram-Manesh, "Organizational changes needed in disasters and public health emergencies: A qualitative study among managers at a major hospital," *International Journal of Disaster Risk Science*, vol. 13, pp. 481–494, 2022.
- [19] I. D'Alessio, A. Quagliari, J. Burrai, A. Pizzo, E. Mari, U. Aitella, G. Lausi, G. Tagliaferri, P. Cordellieri, A. Giannini, and C. Cricenti, "Leading through crisis: A systematic review of institutional decision-makers in emergency contexts," *Behavioral Sciences*, vol. 14, no. 6, p. 481, 2024.
- [20] C. Richardson, *Microservices Patterns: With Examples in Java*. Manning Publications, 2018.
- [21] P. Helland, "Life beyond distributed transactions: An apostate's opinion," *Communications of the ACM*, vol. 60, no. 2, pp. 46–54, 2017.
- [22] L. Lamport, "The part-time parliament," *ACM Transactions on Computer Systems*, vol. 16, no. 2, pp. 133–169, 1998.
- [23] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *Proceedings of the 2014 USENIX Annual Technical Conference*, 2014, pp. 305–320.
- [24] M. Balakrishnan, D. Malkhi, T. Wobber *et al.*, "Tango: Distributed data structures over a shared log," in *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP)*, 2013, pp. 325–340.
- [25] R. van Renesse and F. B. Schneider, "Chain replication for supporting high throughput and availability," in *6th Symposium on Operating Systems Design and Implementation (OSDI 04)*. San Francisco, CA: USENIX Association, Dec. 2004, pp. 91–104. [Online]. Available: <https://www.usenix.org/conference/osdi-04/chain-replication-supporting-high-throughput-and-availability>
- [26] P. S. Almeida, A. Shoker, and C. Baquero, "Efficient state-based CRDTs by delta-mutation," in *International Conference on Networked Systems (NETYS)*, 2014.
- [27] Ø. Hanssen, "Data replication for distributed emergency management systems," in *Norsk IKT-konferanse for forskning og utdanning (NIK)*, 2025.
- [28] P. S. Almeida, "Approaches to conflict-free replicated data types," *ACM Computing Surveys*, vol. 57, no. 2, pp. 1–36, 2024.
- [29] Y. Mao, G. Zhang, Z. Liu, P. Nasirifard, S. Tijanic, and H.-A. Jacobsen, "Making CRDTs not so eventual," *Proceedings of the VLDB Endowment*, vol. 18, no. 2, pp. 349–362, 2024.
- [30] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, "Local-first software: You own your data, in spite of the cloud," in *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, 2019.
- [31] M. Jahanian and K. K. Ramakrishnan, "CoNICE: Consensus in intermittently-connected environments by exploiting naming with application to emergency response," *IEEE/ACM Transactions on Networking*, vol. 30, no. 4, pp. 1926–1939, 2022.
- [32] A. Kumbhar, F. Koohifar, I. Guvenc, and B. Mueller, "A survey on legacy and emerging technologies for public safety communications," *IEEE Communications Surveys & Tutorials*, 2015.
- [33] M. F. H. Sagor, A. Haroon, R. Stoleru, S. Bhunia, A. Altaaweel, M. Chao, L. Jin, M. Maurice, and R. Blalock, "DistressNet-NG: A resilient data storage and sharing framework for mobile edge computing in cyber-physical systems," *ACM Transactions on Cyber-Physical Systems*, vol. 8, no. 3, pp. 37:1–37:31, 2024.
- [34] M. Kleppmann, *Designing Data-Intensive Applications*. O'Reilly Media, 2017.
- [35] C. Li, D. Porto, A. Clement, J. Gehrke, N. Preguiça, and R. Rodrigues, "Making geo-replicated systems fast as possible, consistent when necessary," in *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI 12)*. Hollywood, CA: USENIX Association, 2012, pp. 265–278. [Online]. Available: <https://www.usenix.org/conference/osdi12/technical-sessions/presentation/li>